

Introduction to Express.js

A Fast, Unopinionated & Minimalist Web Framework for Node.js

RAMAGIRI MANISH | Roll No: 25MVCAIR0069



Core Concepts

Understanding the foundation of modern JavaScript backend
development

What is Express.js?

- Express is a flexible, light-weight web framework built on top of Node.js.
- Designed to build fast, scalable, and robust Web Applications and REST APIs.
- Provides higher-level abstractions over raw Node.js HTTP modules.
- Forms the core backend component ('E') in MERN and MEAN technology stacks.



Node.js vs Express.js

Node.js Runtime

An open-source server runtime environment executing JavaScript code server-side.

Uses a low-level HTTP module requiring manual URL parsing, header configuration, and custom routing logic.

Express Framework

A web framework built specifically on top of Node.js to streamline server development.

Includes built-in routing, middleware support, error handling, and clean request/response abstractions.

Key Features & Benefits



Fast & Minimalist

Delivers incredible speed with minimal abstraction layers, giving developers maximum control over the server environment.



Robust Routing

Simplifies endpoint handling with dynamic URL parameters, query parameters, and regex matching for RESTful APIs.



Middleware Chain

Offers a customizable processing pipeline for tasks like authentication, logging, CORS, and data parsing.



Middleware & Routing

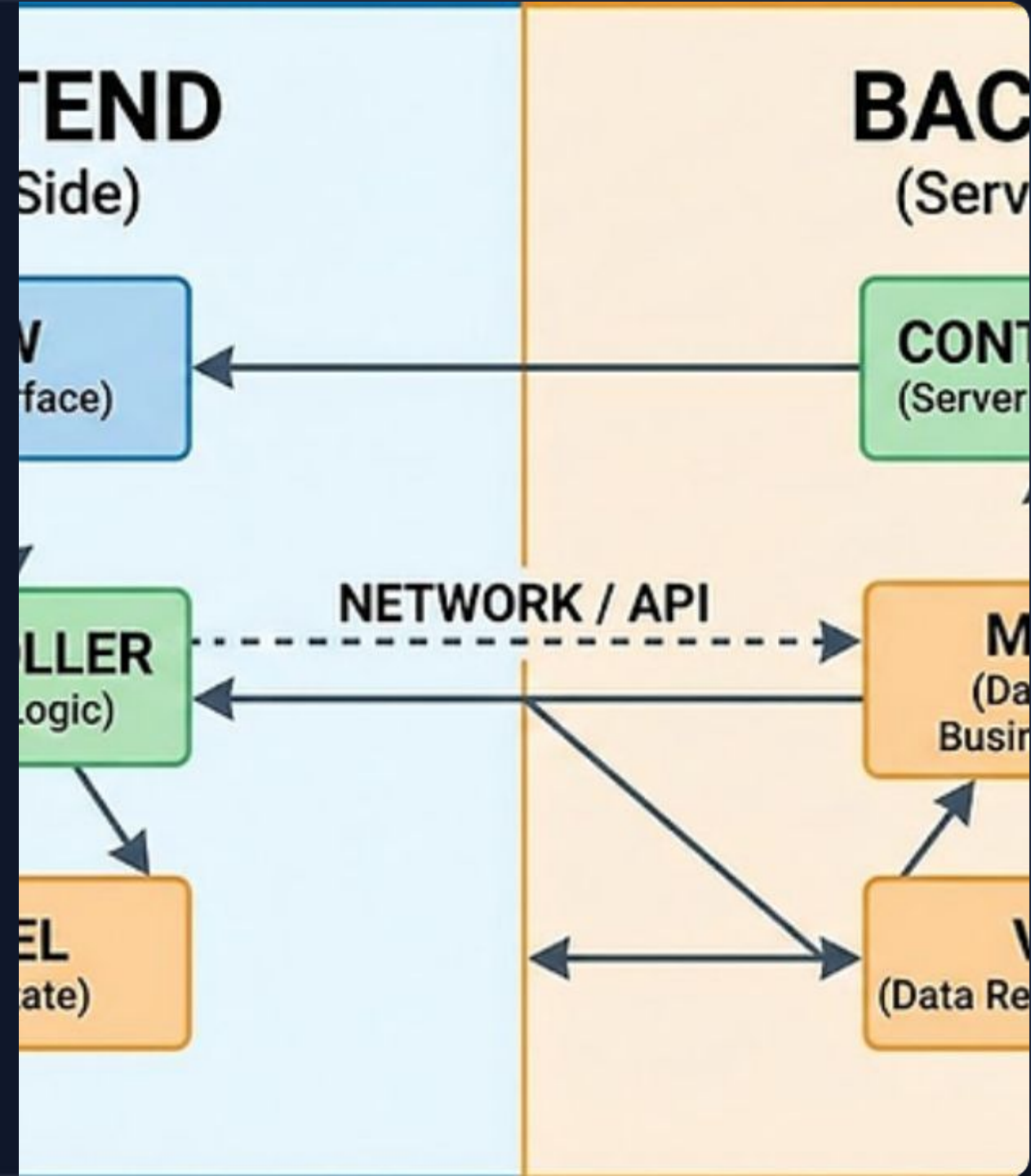
Exploring the core architectural building blocks of Express applications

Understanding Middleware

The Request Pipeline

Middleware functions are functions that have access to the request object (req), the response object (res), and the next function in the cycle.

They can execute code, modify request/response objects, and the response cycle, or pass control to the next handler by calling `next()`.



Types of Middleware



Application-Level: Bound globally to app instance using `app.use()` for app-wide tasks like logging.



Router-Level: Bound to specific router instances using `express.Router()` to scope specific API modules.



Built-In Middleware: Native helpers like `express.json()`, `express.urlencoded()`, and `express.static()`.



Third-Party Middleware: Community packages like `cors`, `helmet`, and `morgan` for security and utilities.

HTTP Methods & Routing

5+

Core HTTP Verbs

RESTful API Endpoint Control

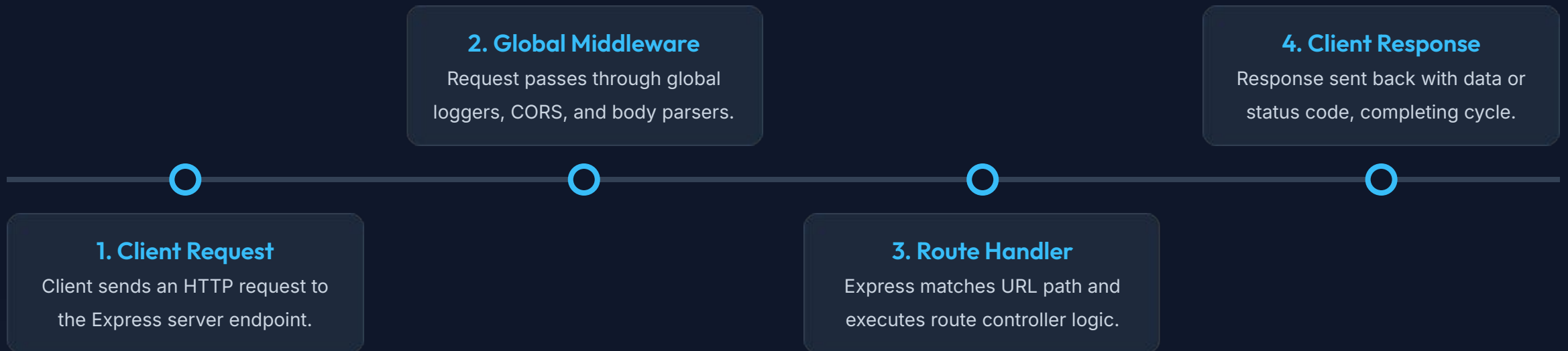
Express maps incoming HTTP requests directly to handler functions using intuitive methods like `app.get()`, `app.post()`, `app.put()`, `app.delete()`, and `app.patch()`.

Routes match endpoint paths and execute dedicated callback functions to process business logic cleanly.

Request & Response Handling

Object	Property / Method	Purpose & Description
req	req.params	Captures route parameters specified in the URL path (e.g. /users/:id).
req	req.query	Extracts URL query string parameters submitted after the ? symbol.
req	req.body	Contains key-value pairs of data submitted in the HTTP request payload.
res	res.json()	Sends a properly formatted JSON response to the requesting client.
res	res.status()	Sets the HTTP status code (e.g. 200 OK, 404 Not Found, 500 Internal Error).

Application Request Lifecycle



Thank You!

Questions & Answers Session

RAMAGIRI MANISH | Roll No: 25MVCAIR0069